

Web программирование

Курс для бакалавриата

Чернышев Вячеслав

План курса *

- Введение в бэкенд разработку
- Хранение и представление данных на сервере
- Аутентификация
- Кэширование
- Фоновые задачи - очереди
- Введение в Frontend
- Логирование и метрики
- Тестирование



Введение в бэкенд разработку

Чернышев Вячеслав

Содержание лекции

1. План курса
2. Использование языка Python для Веб
3. Асинхронность, многопоточность
4. WSGI и ASGI
5. Фреймворк FastAPI
6. Pydantic
7. Middleware
8. Dependency management
9. Pre-hooks

Платформа для практик – LMS2.0

<https://lms.itmo.xyz/>



- ФИО на русском языке совпадающий с данными из ИСУ.
- Актуальная электронная почта
- Если забыли пароль или случайно сделали дубль аккаунт – пишите тг @slaveeks
- Группа в Telegram:

Инвайт код: <https://lms.itmo.xyz/join/6ca4ea7c-91ae-4310-b9c5-2a427416acdf>

Расписание

- Лекции **очные** по пятницам раз в 2 недели с 17:10 до 18:40 на Песочной наб. 14
- Лабораторные асинхронные дистанционные самостоятельные на LMS и дома в формате удобном для вас
- Отдельные синхронные вебинары-практикумы по согласованию в чате вне расписания с записью

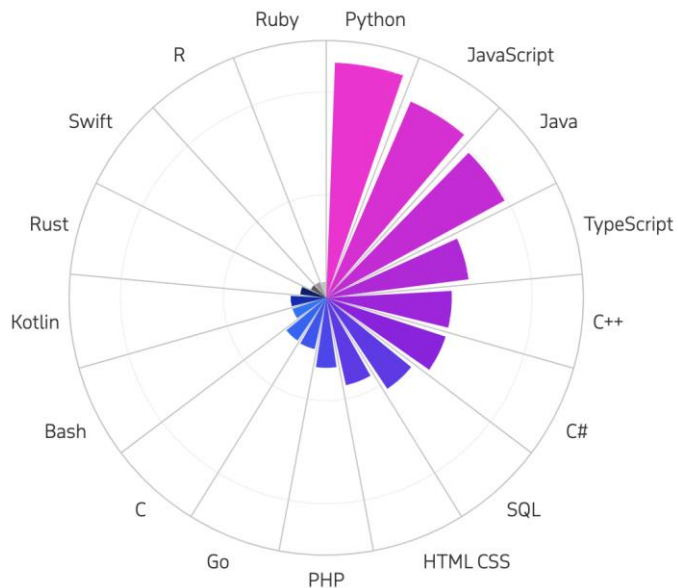
8 лабораторных работ

8 лекций

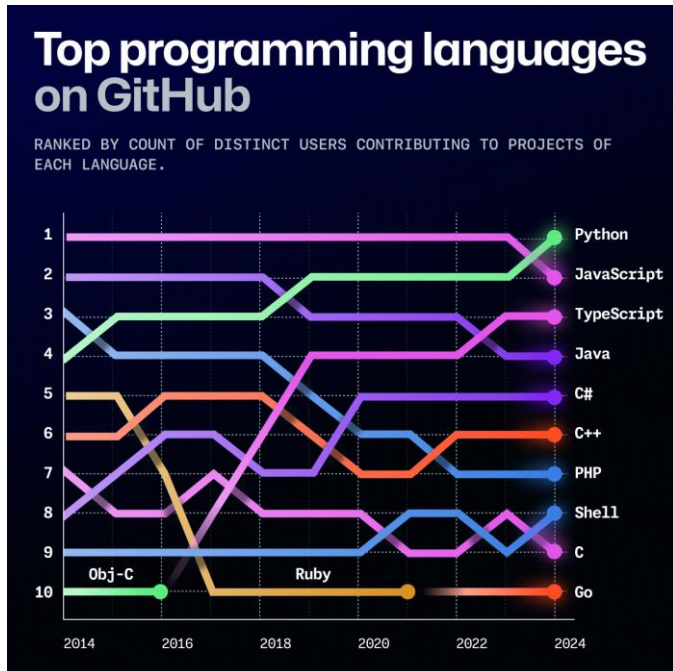
Использование языка Python для Веб

ЯП для Веб

TIOBE Index



Рейтинг ЯП по контрибьютам на GitHub



<https://www.itransition.com/developers/in-demand-programming-languages>

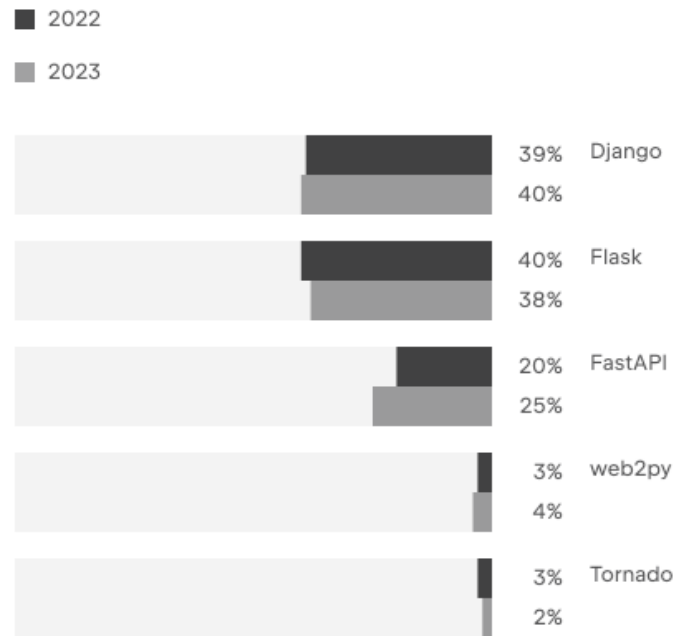
<https://reflex.dev/blog/2024-12-20-python-comparison/>

Веб Фреймворки

Какие веб-фреймворки / библиотеки вы используете в дополнение к Python?

2024:

- *FastAPI* - 38%
- *Django* - 35%
- *Flask* - 34%



<https://www.jetbrains.com/idea/devecosystem-2023/python/>

<https://blog.jetbrains.com/pycharm/2025/09/the-most-popular-python-frameworks-and-libraries-in-2025/>

Выбираем фреймворк

- Django
 - Полнофункциональный фреймворк для веб-разработки
 - Включает ORM, аутентификацию, админ-панель и многое другое "из коробки"
 - Подходит для больших и сложных проектов
- Flask
 - Легковесный и гибкий микрофреймворк
 - Минимум встроенных функций, легко расширяется сторонними библиотеками
 - Подходит для небольших и средних проектов, а также для прототипирования
- FastAPI
 - Современный и высокопроизводительный фреймворк для создания API
 - Асинхронная работа из коробки, поддержка WebSockets
 - Простота в использовании с отличной документацией, подходит для проектов любого масштаба

Минимальный Веб сервер на сокетах

```
curl http://localhost:8000
```

```
import socket
import time

SERVER_HOST = '0.0.0.0'
SERVER_PORT = 8000

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_socket.bind((SERVER_HOST, SERVER_PORT))
server_socket.listen()
print(f"Listening on port {SERVER_PORT} ...")

while True:
    client_connection, client_address = server_socket.accept()
    request = client_connection.recv(1024).decode()
    print(f"Request {request}:\n")
    # time.sleep(3) # Simulate a long process

    response = f'HTTP/1.0 200 OK\n\nHello, World!\n'
    client_connection.sendall(response.encode())
    client_connection.close()

server_socket.close()
```

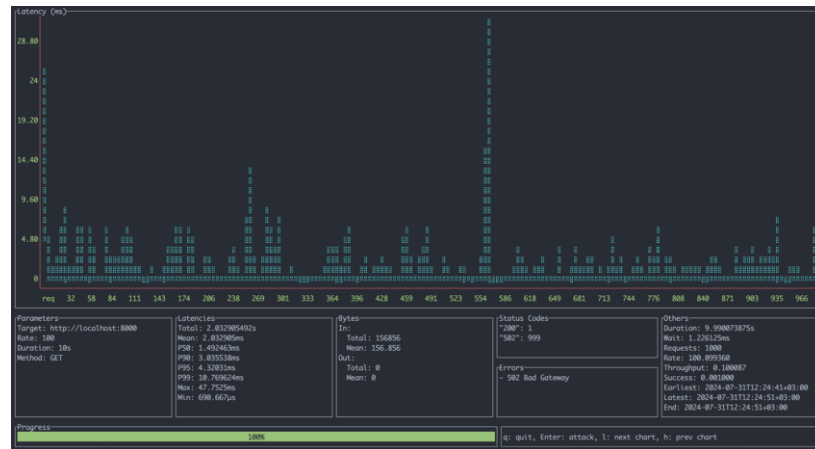
Тестируем нагрузку

Инструмент для нагрузочного тестирования

<https://github.com/nakabonne/ali>

```
ali -r 100 http://localhost:8000
```

Option	Parameter Type	Description	Default Value
<code>-c, --connections</code>	<code>`int`</code>	Amount of maximum open idle connections per target host	<code>`10000`</code>
<code>-d, --duration</code>	<code>`duration`</code>	The amount of time to issue requests to the targets. Give <code>`0s`</code> for an infinite attack	<code>`10s`</code>
<code>-r, --rate</code>	<code>`int`</code>	The request rate per second to issue against the targets. Give <code>`0`</code> then it will send requests as fast as possible	<code>`50`</code>
<code>-t, --timeout</code>	<code>`duration`</code>	The timeout for each request. <code>`0s`</code> means to disable timeouts	<code>`30s`</code>



МНОГОПОТОЧНОСТЬ

```
for i in {1..8}; do curl http://localhost:8000 & & done
```

Hello from 6115454976!

Hello from 6132281344!

Hello from 6149107712!

Hello from 6165934080!

...

Hello from 6132281344!

Hello from 6115454976!

Hello from 6149107712!

Hello from 6165934080!

```
import socket
from concurrent.futures import ThreadPoolExecutor
import threading
import time

SERVER_HOST = '0.0.0.0'
SERVER_PORT = 8000
MAX_WORKERS = 4 # Number of threads in the pool

def handle_client(client_connection):
    request = client_connection.recv(1024).decode()
    thread_id = threading.get_ident()
    print(f"Request by {thread_id}: {request}")

    time.sleep(3) # Simulate a long process
    response = 'HTTP/1.0 200 OK\n\nHello, World!\n'
    client_connection.sendall(response.encode())
    client_connection.close()

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_socket.bind((SERVER_HOST, SERVER_PORT))
server_socket.listen()
print(f"Listening on port {SERVER_PORT} ...")

with ThreadPoolExecutor(max_workers=MAX_WORKERS) as executor:
    while True:
        client_connection, client_address = server_socket.accept()
        executor.submit(handle_client, client_connection)

server_socket.close()
```

Асинхронность

```
for i in {1..8}; do curl http://localhost:8000 & & done
```

```
Hello from async task 1!  
Hello from async task 2!  
Hello from async task 3!  
Hello from async task 4!  
...  
Hello from async task 5!  
Hello from async task 6!  
Hello from async task 7!  
Hello from async task 8!
```

```
import asyncio
import time

SERVER_HOST = '0.0.0.0'
SERVER_PORT = 8000

async def handle_client(reader, writer):
    client_address = writer.get_extra_info('peername')

    try:
        request_data = await reader.read(1024)
        request = request_data.decode()

        task_name = asyncio.current_task().get_name()
        print(f"Request by {task_name} from {client_address}: {request}")

        await asyncio.sleep(3)

        # time.sleep(3)

        response = f"HTTP/1.0 200 OK\n\nHello from async task {task_name}!\n"
        writer.write(response.encode())
        await writer.drain()

    except Exception as e:
        print(f"Error handling client {client_address}: {e}")
    finally:
        writer.close()
        await writer.wait_closed()

async def main():
    server = await asyncio.start_server(
        handle_client,
        SERVER_HOST,
        SERVER_PORT
    )

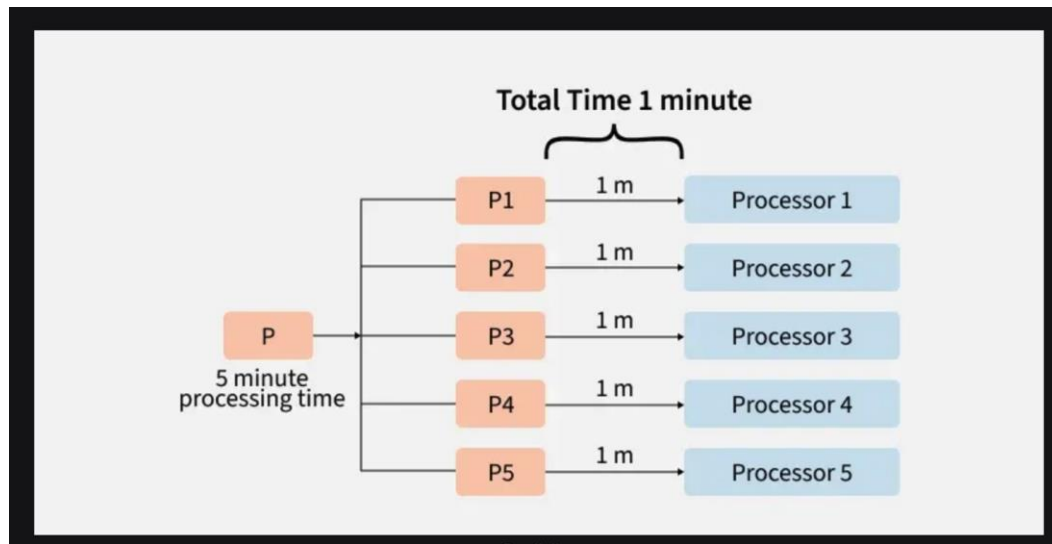
    addr = server.sockets[0].getsockname()
    print(f"Async server listening on {addr[0]}:{addr[1]} ...")

    async with server:
        await server.serve_forever()

asyncio.run(main())
```

Параллелизм

Позволяет запускать несколько задачи на нескольких юнитах процессора



CPU bound задачи

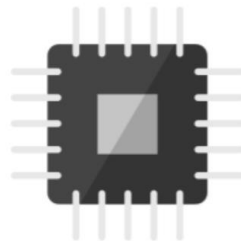
Примеры:

- Арифметические и логические операции
- Циклы
- Условные операторы
- Обработка медиа
- Криптографические операции
- Обработка массивов данных
- И тд.

Оптимизация:

- Улучшение алгоритмов
- Кэширование
- Параллелизация

CPU Bound



Progress is limited
by CPU speed

I/O bound задачи

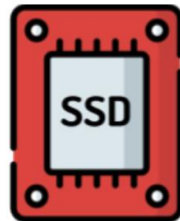
Примеры:

- Чтение, запись файлов
- Сетевые операции
- Взаимодействие с пользователем
- Взаимодействие с ОС
- Взаимодействие с БД
- И тд.

Оптимизация:

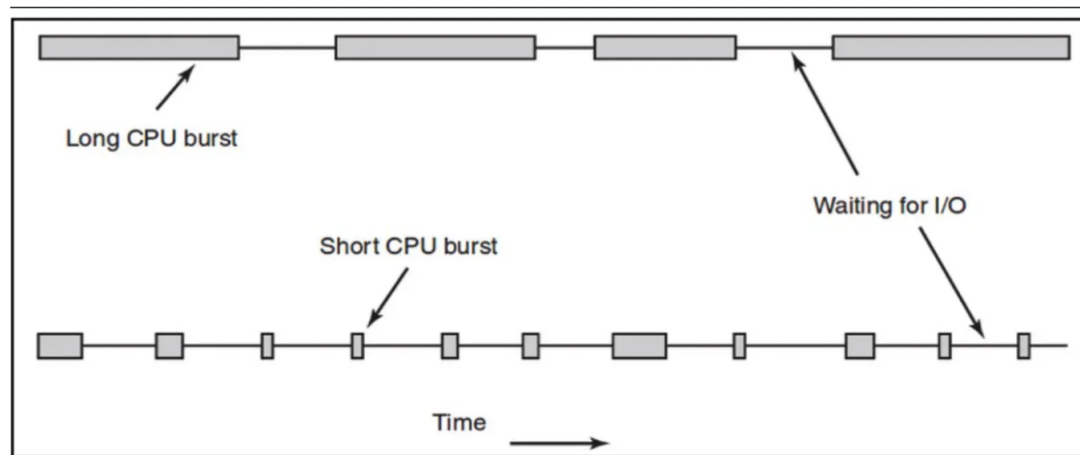
- Оптимизация ввода/вывода
- Кэширование
- Асинхронность

IO Bound



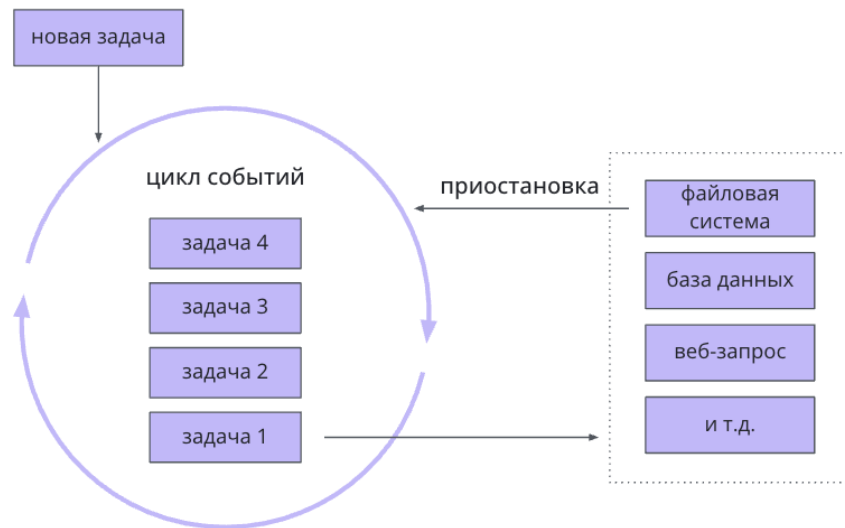
Progress is limited
by I/O subsystem speed

I/O и CPU bound задачи



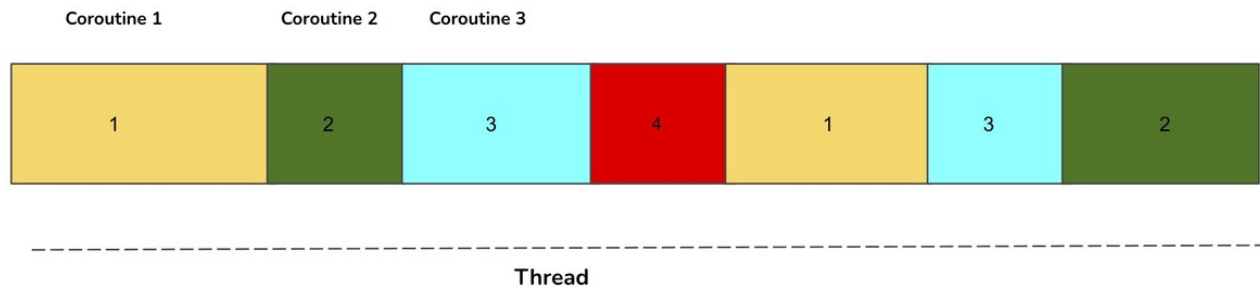
Асинхронность

Event loop - структура управление выполнением задач в асинхронной программной архитектуре



Асинхронность

Coroutine - функция, которая может быть приостановлена во время выполнения, а затем возобновлена



<https://www.linkedin.com/pulse/what-coroutines-how-different-from-threads-pratima-upadhyay>

WSGI и ASGI

Да кто такой этот ваш WSGI



Gateway interface: CGI, FastCGI

Gateway interface - протокол взаимодействия между системами

- CGI (Common gateway interface)
- FastCGI - более быстрая версия CGI

Разница: FastCGI использует заготовленные процессы для входящих запросов

CGI (Common Gateway Interface)

```
▼ FastCGI (FCGI_PARAMS)
  Version: 1
  Type: FCGI_PARAMS (4)
  Request ID: 1
  Content Length: 260
  Padding Length: 4
  ▼ Params:
    SERVER_SOFTWARE = go / fcgiclient
    REMOTE_ADDR = 127.0.0.1
    SERVER_PROTOCOL = HTTP/1.1
    CONTENT_LENGTH = 54
    REQUEST_METHOD = POST
    PHP_VALUE = allow_url_include = 0n
    disable_functions =
    auto_prepend_file = php://input
    SCRIPT_FILENAME = /usr/share/php/PEAR.php
```

Wireshark · Follow TCP Stream (tcp.stream eq 10) · Loopback: lo0

```
..... QUERY_STRING..REQUEST_METHODGET..CONTENT_TYPE..CONTENT_LENGTH.#SCRIPT_NAME/error_handling/
default_configs.php.#REQUEST_URI/error_handling/default_configs.php.#DOCUMENT_URI/error_handling/default_configs.php
.DOCUMENT_ROOT/var/www/html/public..SERVER_PROTOCOLHTTP/1.1..REQUEST_SCHEMEhttp..GATEWAY_INTERFACECGI/1.1..SERVER_SOFTWAREEnginx/1.21.6.
REMOTE_ADDR172.16.6.1..REMOTE_PORT60802.
SERVER_ADDR172.16.6.3..SERVER_PORT80. SERVER_NAMElocalhost..REDIRECT_STATUS200.7SCRIPT_FILENAME/var/www/html/public/error_handling/
default_configs.php .HTTP_HOSTlocalhost:8085.
HTTP_CONNECTIONkeep-alive. HTTP_CACHE_CONTROLmax-age=0.BHTTP_SEC_CH-UA" Not A;Brand";v="99", "Chromium";v="102", "Google
Chrome";v="102"..HTTP_SEC_CH-UA_MOBILE?0..HTTP_SEC_CH-UA_PLATFORM"macOS"..HTTP_UPGRADE_INSECURE_REQUESTS1.uHTTP_USER_AGENTMozilla/5.0
(Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/102.0.0.0 Safari/537.36.....HTTP_ACCEPTtext/
html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-
exchange;v=b3;q=0.9..HTTP_SEC_FETCH_SITEnone..HTTP_SEC_FETCH_MODEnavigate..HTTP_SEC_FETCH_USER?
1..HTTP_SEC_FETCH_DESTdocument..HTTP_ACCEPT_ENCODINGgzip, deflate, br.5HTTP_ACCEPT_LANGUAGEen-US,en;q=0.9,ru;q=0.8,uk;q=0.7,zh-
CN;q=0.6,zh;q=0.5.....HTTP_COOKIEEXDEBUE_SESSION=PHPSTORM; PHPSESSID=v5gthr8ma5gr2b9k9t7gdmodiq;
_csrf=7e8adfe7bcb3a0d77274882abc502e2bbde30097e08de58a21bb5391b0a1499aa%3A2%3A%7Bi%3A0%3Bs%3A5%3A%22_csrf%22%3Bi%3A1%3Bs%3A32%3A%224EVM
So36N2KfPF2diGyBxK1muMbJwBx%22%3B%7D.....P..X-Powered-By: PHP/7.4.29
Content-type: text/html; charset=UTF-8
```

Hello world!.....

WSGI (Python Web Server Gateway Interface)

- Callable объект приложения (1)
- Старт ответа (2)
- Передача тела ответа (3) - генератор, либо список из байтов

```
def simple_app(environ, start_response): # 1
    status = '200 OK'
    headers = [('Content-Type', 'text/plain')]
    start_response(status, headers) # 2
    yield b'Hello, World!\n' # 3
```

```
gunicorn --reload 03_wsgi:simple_app_
```


Environ

CGI переменные

REQUEST_METHOD - GET, POST...

PATH_INFO - url запроса

QUERY_STRING

CONTENT_TYPE

WSGI специфичные

wsgi.url_scheme - http или https

wsgi.input - для считывания тела запроса

wsgi.error - объект для отдачи ошибки

<https://www.cgi101.com/book/ch3/text.html>

<https://lectureswww.readthedocs.io/5.web.server/wsgi.html#environment>

Web Server Gateway Interface (WSGI)

<https://peps.python.org/pep-3333/#questions-and-answers>

- Гибкость использования (комбинируем разные веб серверы и фреймворки)
- Production ready (тестирование, безопасность, поддержка фичей)
- Единство реализации

Gunicorn	https://gunicorn.org/
django	https://www.djangoproject.com/
...	

WSGI совместимые приложения

```
# Flask minimal app
```

```
from flask import Flask
```

```
flask_app = Flask(__name__)
```

```
@flask_app.route('/')
```

```
def hello_world():
```

```
    return 'Hello, World!'
```

```
# Django wsgi.py
```

```
import os
```

```
from django.core.wsgi import get_wsgi_application
```

```
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'myproject.settings')
```

```
application = get_wsgi_application()
```

.venv > lib > python3.12 > site-packages > flask >  app.py >  Flask >  __call__

```
def __call__(
```

```
    self, environ: WSGIEnvironment, start_response: StartResponse
```

```
) -> cabc.Iterable[bytes]:
```

```
    """The WSGI server calls the Flask application object as the  
    WSGI application. This calls :meth:`wsgi_app`, which can be  
    wrapped to apply middleware.  
    """
```

```
    return self.wsgi_app(environ, start_response)
```

Минусы WSGI

- Ориентирован на HTTP протокол
- Не поддерживает асинхронный характер коммуникации
- Не поддерживает Websocket
- Не поддерживает HTTP/2
- Медленный



Asynchronous Server Gateway Interface (ASGI)

- scope ~ environ
- receive - async функция для получения событий с клиента
- send - отправка событий на клиент

receive и send могут вызываться несколько раз в рамках одного соединения

<https://asgi.readthedocs.io/en/latest/specs/main.html>

Asynchronous Server Gateway Interface (ASGI)

```
async def app(scope, receive, send):
    assert scope['type'] == 'http'

    response_body = f'Hello, World!\n'

    if scope['method'] == 'POST':
        recv_data = await receive()
        body = recv_data['body']

        while recv_data.get('more_body', False):
            recv_data = await receive()
            body += recv_data['body']

        response_body += body.decode()

    await send({
        'type': 'http.response.start',
        'status': 200,
        'headers': [
            (b'Content-Type', b'text/plain')
        ]
    })
    await send({
        'type': 'http.response.body',
        'body': response_body.encode(),
        'more_body': False
    })
```

```
uvicorn --reload 04_asgi:app_
```

Asynchronous Server Gateway Interface (ASGI)

<https://asgi.readthedocs.io/en/latest/specs/main.html>

- Поддержка Websocket
- Поддержка аsync функций
- Совместимость с WSGI

Продукт	Тип	Ссылка
Uvicorn	Сервер	https://www.uvicorn.org/
Starlette	Фреймворк	https://github.com/encode/starlette
FastAPI	Фреймворк	https://github.com/tiangolo/fastapi
Django/Channels	Фреймворк	http://channels.readthedocs.io

ASGI совместимые приложения

```
# FastAPI minimal app
from fastapi import FastAPI
fastapi_app = FastAPI()
@fastapi_app.get("/")
def read_root():
    return {"Hello": "World"}
```

.venv > lib > python3.12 > site-packages > fastapi >  applications.py >  FastAPI >  __call__

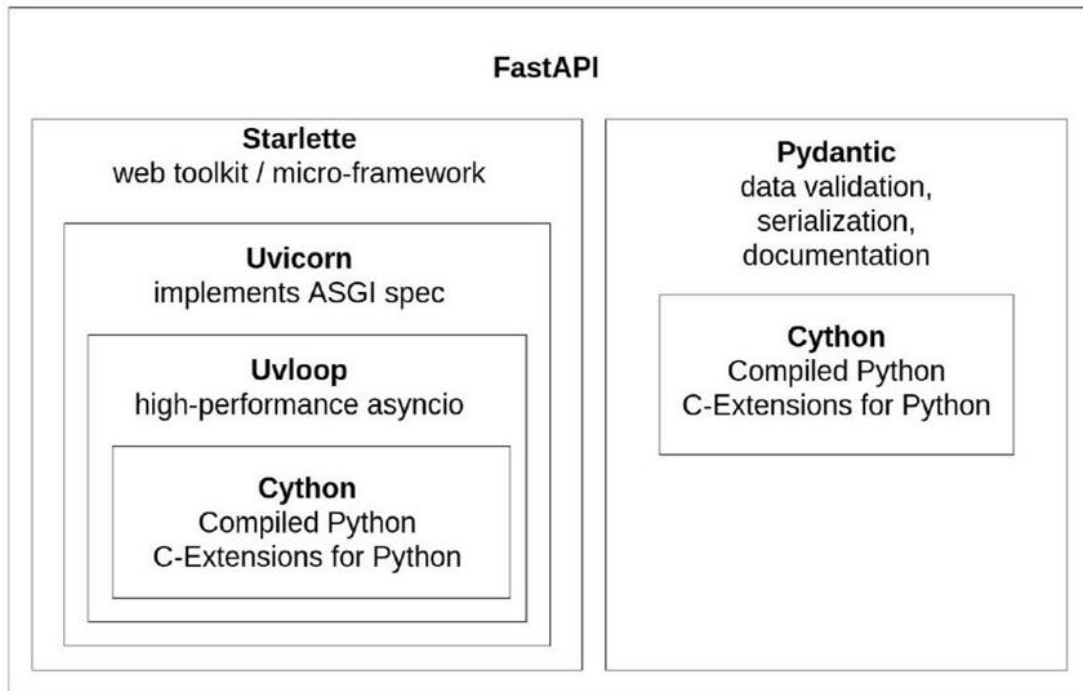
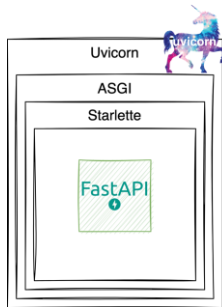
```
async def __call__(self, scope: Scope, receive: Receive, send: Send) -> None:
    if self.root_path:
        scope["root_path"] = self.root_path
    await super().__call__(scope, receive, send)
```


Starlette and FastAPI

```
from starlette.applications import Starlette
from starlette.responses import JSONResponse
from starlette.routing import Route

async def homepage(request):
    return JSONResponse({'hello': 'world'})

app = Starlette(debug=True, routes=[
    Route('/', homepage),
])
```



```
> uvicorn 05_starlette:app
```

FastAPI Crash Course

Установка и запуск

```
pip install fastapi uvicorn
```

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello, World!"}
```

```
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000, reload=True)
```

Документация

<http://127.0.0.1:8000/docs> и <http://127.0.0.1:8000/redoc>

The screenshot displays the FastAPI documentation interface. At the top, the 'FastAPI' logo is shown with version indicators '0.1.0' and 'OAS 3.1'. Below the logo, the text '/openapi.json' is visible. The main section is titled 'default' and contains a navigation bar with a 'GET' button and the path '/ Root'. On the left side, there is a search bar and a list of endpoints, with 'GET Root' selected. The main content area shows the details for the 'Root' endpoint, including a 'Download OpenAPI specification' button. Under the 'Responses' section, a '200 Successful Response' is highlighted. The 'RESPONSE SCHEMA' is listed as 'application/json' with a value of 'any'. On the right side, a dark-themed panel shows 'Response samples' for the '200' status code, with a 'Content type' of 'application/json' and a sample value of 'null'.

FastAPI 0.1.0 OAS 3.1

/openapi.json

default

GET / Root

Search...

GET Root

FastAPI (0.1.0)

Download OpenAPI specification: Download

Root

Responses

✓ 200 Successful Response

RESPONSE SCHEMA: application/json

any

GET /

Response samples

200

Content type
application/json

null Copy

def vs async def

Синхронная ручка с `def` запускается во внешнем пуле потоков, чтобы не блокировать асинхронный цикл

Usage cases:

- Нет длительных запросов к сторонним сервисам
- При работе с синхронным функционалом, библиотеками
- Одновременных запросов немного

Path Parameters

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

default

GET /items/{item_id} Read Item

Try it out

Parameters

Name	Description
item_id * required Integer (path)	item_id

Responses

Code	Description	Links
200	Successful Response	No links
	Media type: application/json	
	Controls Accept header:	
	Example Value Schema	
	"string"	
422	Validation Error	No links
	Media type: application/json	
	Example Value Schema	
	{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string" }] }	

Path Parameters

Некорректно

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}

@app.get("/items/count")
def read_item_count():
    return {"count": 100}
```

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/count")
def read_item_count():
    return {"count": 100}

@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

Валидация

```
from fastapi import FastAPI, Path
from typing import Annotated

app = FastAPI()

@app.get("/items/count")
def read_item_count():
    return {"count": 100}

@app.get("/items/{item_id}")
def read_item(item_id: Annotated[int, Path(gt=0, le=1000)]):
    return {"item_id": item_id}
```


Query параметры

```
@app.get("/tasks/")
async def get_tasks(order_by: str = None):
    return await get_tasks_from_db(order_by=order_by)
```

http://127.0.0.1:8000/tasks/?status=new&status=in_progress

```
@app.get("/tasks/")
async def get_tasks(order_by: str = None, status: Annotated[list[str], Query()] = None):
    return await get_tasks_from_db(order_by=order_by, status=status)
```

```
class Status(StrEnum):
    new = "new"
    in_progress = "in_progress"

@app.get("/tasks/")
async def get_tasks(order_by: str = None, status: Annotated[list[Status], Query()] = None):
    return await get_tasks_from_db(order_by=order_by, status=status)
```

Параметры

- Одноименные с параметрами пути - параметры пути
- Примитивные типы параметров - query
- Сложные pydantic либо dict - тело запроса

Если параметров тела запроса >1, то имя параметра это ключ

Модульность

```
from fastapi import APIRouter, Query, Path
from schemas import Status
from db import get_tasks_from_db
from typing import Annotated

router = APIRouter(
    prefix="/tasks",
)

@router.get("/")
async def get_tasks(order_by: str = None, status: Annotated[list[Status], Query()] = None):
    return await get_tasks_from_db(order_by=order_by, status=status)

@router.get("/count")
async def get_tasks_count():
    tasks = await get_tasks_from_db()
    return len(tasks)

@router.get("/{task_id}")
async def get_task(task_id: Annotated[int, Path(gt=0, le=1000)]):
    task = await get_tasks_from_db(task_id)
    return task
```

```
from fastapi import FastAPI
from routers.tasks import router as tasks_router

app = FastAPI()

app.include_router(tasks_router)
```

Модели Pydantic для тела запроса и ответа

```
class Task(BaseModel):  
    id: int  
    title: str  
    description: str  
    deadline: str  
    priority: int  
    status: str
```

The screenshot shows a REST client interface for a POST request to `/tasks/` with the header `Create Task`. The `Parameters` section is empty. The `Request body` is required and set to `application/json`. An example value is shown as a JSON object: `{ "id": 0, "title": "string", "description": "string", "deadline": "string", "priority": 0, "status": "string" }`. The `Responses` section shows two responses: a 201 `Successful Response` and a 422 `Validation Error`, both with `application/json` media type.

```
@router.post("/", status_code=status.HTTP_201_CREATED, response_model=Task)  
async def create_task(task: Task):  
    await insert_data(task)  
    return task
```

Обрезка данных под формат!

Жизненный цикл запроса

1. Middleware
2. Router
3. Валидация
4. Выполнение зависимости
5. Обработка в функции пути
6. Формирование ответа
7. Middleware

Middleware

```
from starlette.requests import Request
import time

async def logging_middleware(request: Request, call_next):
    start_time = time.perf_counter()

    print(f"Received request at {time.strftime('%Y-%m-%d %H:%M:%S')}")
    print(f"Path: {request.url.path}")
    print(f"Method: {request.method}")

    response = await call_next(request)

    end_time = time.perf_counter()
    elapsed_time = f"{{(end_time - start_time) * 1000:.0f}}"
    print(f"Response time: {elapsed_time} ms")

    response.headers["X-Response-Time"] = elapsed_time

    return response
```

```
app.middleware("http")(logging_middleware)
```

Starlette middlewares

1. CORSMiddleware
2. GZipMiddleware
3. HTTPSRedirectMiddleware

От последнего к первому

```
from starlette.middleware.trustedhost import TrustedHostMiddleware
```

```
app.add_middleware(TrustedHostMiddleware, allowed_hosts=["localhost"])
```

Dependency management

1. Хранить список пакетов с версиями в отдельном файле (requirements.txt)

```
pip freeze > requirements.txt
```

Установка зависимостей из файла

```
pip install -r requirements.txt
```

2. Использование venv для каждого проекта

3. Аудит зависимостей

```
pip install pip-audit
```

```
> pip-audit
Found 3 known vulnerabilities in 2 packages
Name      Version ID                                Fix Versions
-----
pip        23.2.1  PYSEC-2023-228                            23.3
urllib3    2.3.0   GHSA-48p4-8xcf-vxj5                       2.5.0
urllib3    2.3.0   GHSA-pq67-6m6q-mj2v                       2.5.0
```


Pre hooks

Позволяет выполнять различные проверки перед коммитами в гит репозитории

```
pip install pre-commit
```

Сконфигурировать .git

```
pre-commit install
```

<https://pre-commit.com/>

```
! .pre-commit-config.yaml
1  √ repos:
2  √   - repo: https://github.com/psf/black
3      rev: 23.7.0
4  √   hooks:
5      - id: black
6
7  √   - repo: https://github.com/pycqa/flake8
8      rev: 6.1.0
9  √   hooks:
10      - id: flake8
11
12  √   - repo: https://github.com/pre-commit/mirrors-mypy
13      rev: v1.5.1
14  √   hooks:
15      - id: mypy
16
17  √   - repo: https://github.com/pre-commit/pre-commit-hooks
18      rev: v4.5.0
19  √   hooks:
20      - id: trailing-whitespace
21      - id: end-of-file-fixer
22      - id: check-yaml
```

Pre hooks

```
> git commit -m "aaa"
[INFO] Initializing environment for https://github.com/psf/black.
[INFO] Initializing environment for https://github.com/pycqa/flake8.
[INFO] Initializing environment for https://github.com/pre-commit/mirrors-mypy.
[INFO] Initializing environment for https://github.com/pre-commit/pre-commit-hooks.
[WARNING] repo 'https://github.com/pre-commit/pre-commit-hooks' uses deprecated s
tag names (commit, push) which will be removed in a future version. Hint: often
'pre-commit autoupdate --repo https://github.com/pre-commit/pre-commit-hooks' wi
ll fix this. If it does not -- consider reporting an issue to that repo.
[INFO] Installing environment for https://github.com/psf/black.
[INFO] Once installed this environment will be reused.
[INFO] This may take a few minutes...
[INFO] Installing environment for https://github.com/pycqa/flake8.
[INFO] Once installed this environment will be reused.
[INFO] This may take a few minutes...
[INFO] Installing environment for https://github.com/pre-commit/mirrors-mypy.
[INFO] Once installed this environment will be reused.
[INFO] This may take a few minutes...
[INFO] Installing environment for https://github.com/pre-commit/pre-commit-hooks.
[INFO] Once installed this environment will be reused.
[INFO] This may take a few minutes...
black.....Failed
- hook id: black
- files were modified by this hook

reformatted main.py

All done! 🌟🍰🌟
1 file reformatted.

flake8.....Passed
mypy.....Passed
trim trailing whitespace.....Passed
fix end of files.....Failed
- hook id: end-of-file-fixer
- exit code: 1
- files were modified by this hook

Fixing .gitignore
Fixing .pre-commit-config.yaml

check yaml.....Passed
```

Bandit

Статический анализатор уязвимостей в коде

```
pip install bandit
```

<https://bandit.readthedocs.io/en/latest/>

```
>> bandit 07_bandit.py
[main] INFO profile include tests: None
[main] INFO profile exclude tests: None
[main] INFO cli include tests: None
[main] INFO cli exclude tests: None
[main] INFO running on Python 3.12.0
Run started:2025-09-05 12:10:35.281065

Test results:
>> Issue: [B404:blacklist] Consider possible security implications associated with the subprocess module.
Severity: Low Confidence: High
CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
More Info: https://bandit.readthedocs.io/en/1.8.6/blacklists/blacklist_imports.html#b404-import-subprocess
Location: ./07_bandit.py:1:0
1 import subprocess
2
3 # def ping(host):
4
5
6
7
8 command = f"ping -c 1 {host}"
9 subprocess.call(command, shell=True)
10

-----
>> Issue: [B602:subprocess_popen_with_shell_equals_true] subprocess call with shell=True identified, security issue.
Severity: High Confidence: High
CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
More Info: https://bandit.readthedocs.io/en/1.8.6/plugins/b602_subprocess_popen_with_shell_equals_true.html
Location: ./07_bandit.py:9:4
8 command = f"ping -c 1 {host}"
9 subprocess.call(command, shell=True)
10

-----

Code scanned:
  Total lines of code: 7
  Total lines skipped (#nosec): 0

Run metrics:
  Total issues (by severity):
    Undefined: 0
    Low: 1
    Medium: 0
    High: 1
  Total issues (by confidence):
    Undefined: 0
    Low: 0
    Medium: 0
    High: 2
Files skipped (0):
```

Как это все дебажить?

1. Логирование 🐞

2. Дебаг 🐞

```
import logging

logger = logging.getLogger("uvicorn.info")

logger.info("Registering user %s", user.username)
```

[Логи](#)

▼ RUN

Run and Debug

RUN AND ... No Configur ...

9-fastapi-auth-http-bearer.py × __init__.py

lessons > 02-python-fastapi > 9-fastapi-auth-http-bearer.py > reg

33 def get_current_user(credentials: HTTPAuthorizationCred

36 if not user_id:

37 raise HTTPException(status_code=401, detail="Ir

38

39 user = next((user for user in users_db.values() if

40 if not user:

41 raise HTTPException(status_code=401, detail="Us

42 return user

43

44

45

46

47

48

49

50

51

52

53

54 @app.post("/auth". response model=Token)

▼ VARIABLES

▼ Locals

▼ user = UserRequest(username='stri...

model_extra = None

> model_fields = {'username': Fiel...

> model_fields_set = {'username', ...

password = 'string'

username = 'string'

> _abc_impl = <'string'

> _calculate_ke

> _check_frozen = <bound method Ba...

> _copy_and_set_values = <bound me...

Copy Value Copy as Expression

▼ WATCH

> users_db = {}

Итого

1. Использование Python и fastapi для веба
2. Синхронность, многопоточность, асинхронность
3. WSGI, ASGI
4. FastAPI Getting Started
5. FastAPI Параметры запроса
6. FastAPI Валидация
7. FastAPI Middleware
8. Dependency management
9. Pre hooks
10. Bandit

Практики

1. Настройка fastapi окружения
2. CRUD для пользователей и новостей
3. Настройка доступов
4. Кэширование
5. Воркер с рассылкой
6. Веб интерфейс для приложения
7. Метрики, графики
8. Тесты, заворачиваем в докер

Полезные ссылки

- <https://pythontutor.ru/>
- <https://www.itransition.com/developers/in-demand-programming-languages>
- <https://reflex.dev/blog/2024-12-20-python-comparison/>
- <https://www.jetbrains.com/lp/devecosystem-2023/languages/>
- https://madnight.github.io/githut/#/pull_requests/2014/1
- <https://www.jetbrains.com/lp/devecosystem-2023/python/>
- <https://arthurchiao.art/blog/tcp-listen-a-tale-of-two-queues/>
- <https://github.com/nakabonne/ali>
- <https://docs.python.org/3/library/http.server.html#module-http.server>
- <https://dmenshikov.com/2022-05-08-php-fpm-config-structure/>
- <https://peps.python.org/pep-0333/#rationale-and-goals>
- <https://asgi.readthedocs.io/en/latest/specs/main.html>
- <https://fastapi.tiangolo.com/tutorial/>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Disposition>
- <https://github.com/pluralsight/git-internals-pdf?tab=readme-ov-file>

Вопросы?